

Simulación Efecto Diodo Superconductor — Peine Asimétrico

Paralelizado | 64 núcleos

Variables:

- N dientes del peine: valor fijo configurable en Celda 2
- Campo B: 0.0 a 1.2 mT (pasos de 0.1)
- Polaridades: positivo y negativo (invertir source/drain)
- Barrido corriente: 0 a 50 μA (pasos de 0.1)

```
In [ ]: # =====  
# CELDA 1 – IMPORTS  
# =====  
%config InlineBackend.figure_formats = {"retina", "png"}  
%matplotlib inline  
  
import os  
import time  
import tempfile  
import shutil  
import itertools  
import threading  
import traceback  
import glob  
from datetime import datetime  
  
import numpy as np  
import pandas as pd  
import matplotlib  
matplotlib.use('Agg') # Backend sin pantalla, necesario para paralelización  
import matplotlib.pyplot as plt  
  
from joblib import Parallel, delayed
```

```
import tdgl
from tdgl.geometry import box

print("Imports completados correctamente.")
```

```
In [ ]: # =====
# CELDA 2 – PARÁMETROS GLOBALES
# =====
TEMP_DIR = "/media/david/ea1787b7-95f4-4875-ace4-8e79eb830ea3/home/david/Documents/Problema 3"

# =====
# PARÁMETROS DEL MATERIAL
# =====
XI          = 0.5    # Longitud de coherencia (μm)
LONDON_LAMBDA = 2.0    # Longitud de penetración de London (μm)
D_THICKNESS  = 0.1    # Espesor de la lámina (μm)
GAMMA       = 1.0    # Parámetro de amortiguamiento

# =====
# PARÁMETROS GEOMÉTRICOS FIJOS
# =====
TOTAL_LENGTH = 20.0    # Lx: longitud total del dispositivo (μm)
TOTAL_WIDTH  = 5.0     # Ly: altura del bloque rectangular base (μm)
HD           = 3.5     # Profundidad máxima de los dientes (μm)

# =====
# CONTACTOS
# =====
CONTACT_LENGTH = TOTAL_WIDTH    # Toda la pared lateral (μm)
CONTACT_WIDTH  = 0.1            # Ancho del contacto en x (μm)

# =====
# ★ NÚMERO DE DIENTES DEL PEINE – VALOR FIJO
# Modifica este valor para cambiar la simulación
# =====
N_PEINE = 2    # Número de crestas del peine

# =====
# PARÁMETROS DE SIMULACIÓN
```

```

# -----
B_VALORES = [round(b, 1) for b in np.arange(0.0, 1.3, 0.1)] # Campo magnético externo (mT)
POLARIDADES = ["positivo", "negativo"] # Polaridad de corriente

# -----
# BARRIDO DE CORRIENTE
# -----
CORRIENTE_MIN = -50.0 # Corriente mínima del barrido (μA)
CORRIENTE_MAX = 50.0 # Corriente máxima del barrido (μA)
CORRIENTE_PASO = 1 # Paso del barrido (μA)

# -----
# OPCIONES DEL SOLVER
# -----
SKIP_TIME = 50 # Tiempo de estabilización antes de medir (u.a.)
SOLVE_TIME = 100 # Tiempo de simulación activa (u.a.)

# -----
# PARALELIZACIÓN
# -----
N_JOBS = 65 # Núcleos a usar (de 128 disponibles)

# -----
# CARPETAS Y RUTAS DE RESULTADOS
# Los resultados de este N se guardan en:
# Results/N{N_PEINE}_crestas/
# -----
RESULTS_DIR = "Results"
N_DIR = os.path.join(RESULTS_DIR, f"N{N_PEINE}_crestas")
CSV_PATH = os.path.join(N_DIR, f"resultados_N{N_PEINE}.csv")
TXT_PATH = os.path.join(N_DIR, "parametros_simulacion.txt")

# Lista de trabajos: (B, polaridad) – N es fijo
TRABAJOS = list(itertools.product(B_VALORES, POLARIDADES))

print(f"N_PEINE fijo: {N_PEINE}")
print(f"B_valores: {B_VALORES}")
print(f"Polaridades: {POLARIDADES}")
print(f"Total trabajos: {len(TRABAJOS)}")
print(f"Núcleos a usar: {N_JOBS}")
print(f"Carpeta de salida: {N_DIR}")

```

```

In [ ]: # =====
# CELDA 3 – CREAR ESTRUCTURA DE CARPETAS Y TXT
# =====

def crear_estructura_carpetas():
    """Crea la estructura Results/N{N}_crestas/polaridad/B-x.x/ para el N fijo."""
    for pol in POLARIDADES:
        for B in B_VALORES:
            ruta = os.path.join(N_DIR, pol, f"B-{B:.1f}")
            os.makedirs(ruta, exist_ok=True)
        print(f"Estructura de carpetas creada en: {N_DIR}/")

def guardar_txt_parametros():
    """Guarda un TXT con todos los parámetros de simulación junto al CSV de resultados."""
    with open(TXT_PATH, "w", encoding="utf-8") as f:
        f.write("=" * 55 + "\n")
        f.write("  PARÁMETROS DE SIMULACIÓN\n")
        f.write(f"  Fecha inicio: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n")
        f.write("=" * 55 + "\n\n")

        f.write("---- MATERIAL ---\n")
        f.write(f"xi (coherencia)           = {XI} μm\n")
        f.write(f"london_lambda              = {LONDON_LAMBDA} μm\n")
        f.write(f"d (espesor)                = {D_THICKNESS} μm\n")
        f.write(f"gamma                      = {GAMMA}\n\n")

        f.write("---- GEOMETRÍA FIJA ---\n")
        f.write(f"Lx (longitud total)        = {TOTAL_LENGTH} μm\n")
        f.write(f"Ly (altura bloque)         = {TOTAL_WIDTH} μm\n")
        f.write(f"hd (prof. dientes)         = {HD} μm\n")
        f.write(f"Pared restante (Ly-hd)    = {TOTAL_WIDTH - HD} μm\n")
        f.write(f"contact_length             = {CONTACT_LENGTH} μm (toda la pared)\n")
        f.write(f"contact_width              = {CONTACT_WIDTH} μm\n\n")

        f.write("---- CONFIGURACIÓN SIMULADA ---\n")
        f.write(f"N dientes (fijo)          = {N_PEINE}\n")
        f.write(f"Campo B (mT)              = {B_VALORES}\n")
        f.write(f"Polaridades               = {POLARIDADES}\n")
        f.write(f"Total trabajos            = {len(TRABAJOS)}\n\n")

        f.write("---- BARRIDO DE CORRIENTE ---\n")

```

```

f.write(f"Corriente min          = {CORRIENTE_MIN}  $\mu$ A\n")
f.write(f"Corriente max          = {CORRIENTE_MAX}  $\mu$ A\n")
f.write(f"Paso                    = {CORRIENTE_PASO}  $\mu$ A\n")
f.write(f"Número de puntos        = {len(np.arange(CORRIENTE_MIN, CORRIENTE_MAX, CORRIENTE_PASO))}\n")

f.write("--- SOLVER ---\n")
f.write(f"skip_time                    = {SKIP_TIME}\n")
f.write(f"solve_time                    = {SOLVE_TIME}\n")
f.write(f"max_edge_length                = xi/2 = {XI/2}  $\mu$ m\n")
f.write(f"smooth                        = 100\n\n")

f.write("--- PARALELIZACIÓN ---\n")
f.write(f"CPUs disponibles                = 128\n")
f.write(f"n_jobs                        = {N_JOBS}\n")

print(f"Parámetros guardados en: {TXT_PATH}")

crear_estructura_carpetas()
guardar_txt_parametros()

```

```

In [ ]: # =====
# CELDA 4 – FUNCIÓN: CONSTRUIR DISPOSITIVO
# =====

def construir_dispositivo(N):
    """
    Construye el dispositivo superconductor de peine asimétrico con N dientes.
    El contacto ocupa la totalidad de la pared lateral.
    Retorna el device con malla generada.
    """
    # Parámetros de los dientes gaussianos (Ec. 1 del paper)
    delta_x = TOTAL_LENGTH / N          # Separación entre centros de dientes ( $\mu$ m)
    delta   = 0.20 * delta_x            # Parámetro de suavidad ( $\delta = 0.20 \cdot \Delta x$ )
    x_teeth = np.linspace(
        -TOTAL_LENGTH / 2 + delta_x / 2,
        TOTAL_LENGTH / 2 - delta_x / 2,
        N
    )

    # Perfil del borde inferior con dientes gaussianos
    def lower_edge_profile(x):

```

```

    y = np.zeros_like(x, dtype=float)
    for xi_i in x_teeth:
        y += -HD * np.exp(-((x - xi_i) ** 2) / (2 * delta ** 2))
    return y

# Construcción del polígono
n_pts = 1000
x_bottom = np.linspace(-TOTAL_LENGTH / 2, TOTAL_LENGTH / 2, n_pts)
y_bottom = lower_edge_profile(x_bottom)
y_top = TOTAL_WIDTH / 2
y_base = -TOTAL_WIDTH / 2
y_bottom_coords = y_base + y_bottom

x_polygon = np.concatenate([
    np.array([-TOTAL_LENGTH/2, TOTAL_LENGTH/2]), # borde superior
    np.array([TOTAL_LENGTH/2, TOTAL_LENGTH/2]), # lado derecho
    x_bottom[::-1], # borde inferior (dientes)
    np.array([-TOTAL_LENGTH/2, -TOTAL_LENGTH/2]) # lado izquierdo
])
y_polygon = np.concatenate([
    np.array([y_top, y_top]),
    np.array([y_top, y_base]),
    y_bottom_coords[::-1],
    np.array([y_base, y_top])
])

polygon_points = np.column_stack([x_polygon, y_polygon])

# Film
film = (
    tdgl.Polygon("film", points=polygon_points)
    .resample(1200)
    .buffer(0)
)

# Terminales: contacto ocupa toda la pared lateral
source = (
    tdgl.Polygon("source", points=box(CONTACT_WIDTH, CONTACT_LENGTH))
    .translate(dx=-TOTAL_LENGTH / 2)
)
drain = source.scale(xfact=-1).set_name("drain")

```

```

# Puntos de medición de voltaje
probe_points = [
    (-TOTAL_LENGTH / 2.5, 0),
    (TOTAL_LENGTH / 2.5, 0)
]

# Layer
layer = tdgl.Layer(
    coherence_length=XI,
    london_lambda=LONDON_LAMBDA,
    thickness=D_THICKNESS,
    gamma=GAMMA
)

# Device
device = tdgl.Device(
    f"peine_N{N}",
    layer=layer,
    film=film,
    terminals=[source, drain],
    probe_points=probe_points,
    length_units="um",
)

# Generar malla
device.make_mesh(max_edge_length=XI / 2, smooth=100)

return device

print("Función construir_dispositivo() definida.")

```

```

In [ ]: # =====
# CELDA 5 – FUNCIONES AUXILIARES
# =====

def get_terminal_currents(current, polaridad):
    if polaridad == "positivo":
        return {"source": current, "drain": -current}
    else:
        return {"source": -current, "drain": current}

```

```

def detectar_corrientes_criticas(voltages, currents):
    gaps = [(voltages[i+1] - voltages[i], i) for i in range(len(voltages) - 1)]
    gaps_sorted = sorted(gaps, key=lambda x: x[0], reverse=True)

    # Curva plana: no hay corriente crítica
    rango = max(voltages) - min(voltages)
    if rango < 0.01:
        return None, None

    umbral = rango * 0.05
    saltos_significativos = [(g, i) for g, i in gaps_sorted if g > umbral]

    if len(saltos_significativos) == 0:
        return None, None
    elif len(saltos_significativos) == 1:
        idx1 = saltos_significativos[0][1]
        return currents[idx1], None
    else:
        indices = sorted([s[1] for s in saltos_significativos[:2]])
        return currents[indices[0]], currents[indices[1]]

def guardar_fila_csv(fila_dict, n_dir):
    """Cada proceso escribe su propio archivo parcial dentro de N_DIR – sin lock."""
    pid = os.getpid()
    ruta_parcial = os.path.join(n_dir, f"parcial_{pid}.csv")
    fila_df = pd.DataFrame([fila_dict])
    header = not os.path.exists(ruta_parcial)
    fila_df.to_csv(ruta_parcial, mode='a', header=header, index=False)

print("Funciones auxiliares definidas.")

```

```

In [ ]: # =====
# CELDA 6 – FUNCIÓN PRINCIPAL: CORRER SIMULACIÓN
# =====

```

```

def correr_simulacion(B, polaridad, n_dir=N_DIR):
    """
    Corre una simulación completa para (B, polaridad) con N_PEINE fijo:
    1. Construye el dispositivo
    2. Simulación espacial → mapas de corrientes y  $|\psi|^2$ 
    3. Barrido V-I → datos guardados en CSV + curva
    """

```



```

4. Guarda todas las gráficas y actualiza el CSV parcial
"""
os.environ["OPENBLAS_NUM_THREADS"] = "2"
N = N_PEINE # N fijo desde parámetros globales
label = f"[N={N}], B={B:.1f}mT, {polaridad}]"
t_inicio_total = time.time()

# Carpeta de salida para este trabajo
ruta_salida = os.path.join(N_DIR, polaridad, f"B-{B:.1f}") # ← global, no n_dir

# Carpeta temporal propia para el .h5 (evita colisiones entre procesos)
tmpdir = tempfile.mkdtemp(dir=TEMP_DIR)

try:
    # — Construir dispositivo —
    device = construir_dispositivo(N)

    # — Opciones del solver —
    h5_path = os.path.join(tmpdir, f"sim_N{N}_B{B}_{polaridad}.h5")
    options = tdgl.SolverOptions(
        skip_time=SKIP_TIME,
        solve_time=SOLVE_TIME,
        output_file=h5_path,
        field_units="mT",
        current_units="uA",
        save_every=100,
    )

    # — Campo magnético externo perpendicular —
    campo_B = tdgl.sources.ConstantField(B, field_units="mT")

    # =====
    # 1. SIMULACIÓN ESPACIAL (corriente fija = 12  $\mu$ A)
    # =====
    t0 = time.time()

    solucion_espacial = tdgl.solve(
        device=device,
        options=options,
        applied_vector_potential=campo_B,
        terminal_currents=get_terminal_currents(12.0, polaridad)
    )

```

```

t_espacial = time.time() - t0
print(f"{label} Simulación espacial: {t_espacial:.1f}s")

# Mapa de corrientes
fig, axes = plt.subplots(1, 2, figsize=(12, 4), constrained_layout=True)
_ = solucion_espacial.plot_currents(ax=axes[0], streamplot=False)
_ = solucion_espacial.plot_currents(ax=axes[1])
fig.suptitle(f'Corrientes - N={N}, B={B:.1f}mT, {polaridad}')
fig.savefig(os.path.join(ruta_salida, "mapa_corrientes.png"), dpi=150)
plt.close(fig)

# Densidad de pares de Cooper  $|\psi|^2$ 
fig, axes_op = solucion_espacial.plot_order_parameter()
axes_op.flat[0].set_title(f' $|\psi|^2$  - N={N}, B={B:.1f}mT, {polaridad}')
fig.savefig(os.path.join(ruta_salida, "densidad_cooper.png"), dpi=150)
plt.close(fig)

# =====
# 2. BARRIDO V-I
# =====

t0 = time.time()

corrientes = np.arange(CORRIENTE_MIN, CORRIENTE_MAX, CORRIENTE_PASO)
voltajes = []
solucion_anterior = None

for current in corrientes:
    h5_barrido = os.path.join(tmpdir, "barrido_actual.h5")
    if os.path.exists(h5_barrido):
        os.remove(h5_barrido)

    opts_barrido = tdgl.SolverOptions(
        skip_time=SKIP_TIME,
        solve_time=SOLVE_TIME,
        output_file=h5_barrido,
        field_units="mT",
        current_units="uA",
        save_every=100,
    )
    sol = tdgl.solve(
        device=device,

```

```

        options=opts_barrido,
        applied_vector_potential=campo_B,
        terminal_currents=get_terminal_currents(current, polaridad),
        seed_solution=solucion_anterior
    )
    voltajes.append(sol.dynamics.mean_voltage())
    solucion_anterior = sol

t_barrido = time.time() - t0
print(f"{label} Barrido V-I: {t_barrido:.1f}s")

# — Guardar CSV del barrido V-I en la carpeta del campo B —
csv_vi_path = os.path.join(ruta_salida, "VI_data.csv")
df_vi = pd.DataFrame({'current_uA': corrientes, 'voltage_V': voltajes})
df_vi.to_csv(csv_vi_path, index=False)
print(f"{label} Datos V-I guardados en: {csv_vi_path}")

# — Detectar corrientes críticas —
Jc1, Jc2 = detectar_corrientes_criticas(voltajes, corrientes)

# — Gráfica curva V-I —
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(corrientes, voltajes, color='blue', linewidth=1)
if Jc1 is not None:
    ax.axvline(Jc1, color='red', linestyle='--', label=f'Jc1 = {Jc1:.2f}  $\mu$ A')
if Jc2 is not None:
    ax.axvline(Jc2, color='orange', linestyle='--', label=f'Jc2 = {Jc2:.2f}  $\mu$ A')
ax.set_xlabel('Corriente ( $\mu$ A)')
ax.set_ylabel('Voltaje promedio (V)')
ax.set_title(f'Curva V-I - N={N}, B={B:.1f}mT, {polaridad}')
ax.grid(True)
ax.legend()
plt.tight_layout()
fig.savefig(os.path.join(ruta_salida, "curva_VI.png"), dpi=150)
plt.close(fig)

# =====
# 3. GUARDAR FILA EN CSV PARCIAL
# =====

t_total = time.time() - t_inicio_total

fila = {

```

```

        "N_peine":      N,
        "polaridad":   polaridad,
        "B_mT":        B,
        "Jc1_uA":       Jc1 if Jc1 is not None else np.nan,
        "Jc2_uA":       Jc2 if Jc2 is not None else np.nan,
        "eta_c1_pct":   np.nan,    # se calcula en post-procesamiento
        "eta_c2_pct":   np.nan,    # se calcula en post-procesamiento
        "tiempo_s":     round(t_total, 1),
        "estado":       "OK"
    }
    guardar_fila_csv(fila, n_dir)

    print(f"{label} TOTAL: {t_total:.1f}s | Jc1={Jc1}  $\mu$ A | Jc2={Jc2}  $\mu$ A")
    return fila

except Exception as e:
    t_total = time.time() - t_inicio_total
    print(f"{label} ERROR después de {t_total:.1f}s: {e}")
    traceback.print_exc()

    fila_error = {
        "N_peine":      N_PEINE,
        "polaridad":   polaridad,
        "B_mT":        B,
        "Jc1_uA":       np.nan,
        "Jc2_uA":       np.nan,
        "eta_c1_pct":   np.nan,
        "eta_c2_pct":   np.nan,
        "tiempo_s":     round(t_total, 1),
        "estado":       f"ERROR: {str(e)[:80]}"
    }
    guardar_fila_csv(fila_error, n_dir)
    return fila_error

finally:
    shutil.rmtree(tmpdir, ignore_errors=True)

print("Función correr_simulacion() definida.")

```

```

In [ ]: # =====
        # CELDA 7 – LANZAR PARALELIZACIÓN
        # =====

```

```

print("=" * 55)
print(f" INICIANDO SIMULACIÓN – N_PEINE = {N_PEINE}")
print(f" {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
print(f" Total trabajos: {len(TRABAJOS)} ({len(B_VALORES)} campos × {len(POLARIDADES)} polaridades)")
print(f" Núcleos: {N_JOBS}")
print("=" * 55)

t_N_inicio = time.time()

resultados = Parallel(
    n_jobs=N_JOBS,
    backend="loky",
    verbose=5
)(
    delayed(correr_simulacion)(B, polaridad)
    for B, polaridad in TRABAJOS
)

t_N_total = time.time() - t_N_inicio

print("\n" + "=" * 55)
print(f" SIMULACIÓN COMPLETADA – N_PEINE = {N_PEINE}")
print(f" Tiempo total N={N_PEINE}: {t_N_total:.1f}s")
print("=" * 55)

```

```

In [ ]: # =====
# CELDA 8 – POST-PROCESAMIENTO: CONSOLIDAR Y CALCULAR ETA
# =====

# — Consolidar archivos parciales del N actual —
parciales = glob.glob(os.path.join(N_DIR, "parcial_*.csv"))

if parciales:
    dfs = [pd.read_csv(p) for p in parciales]
    df_consolidado = pd.concat(dfs, ignore_index=True)
    for p in parciales:
        os.remove(p)
    print(f"Consolidados desde {len(parciales)} parciales.")
elif 'resultados' in dir() and resultados:
    filas = [r for r in resultados if r is not None]
    df_consolidado = pd.DataFrame(filas)

```

```
        print(f"Usando variable 'resultados' directamente ({len(filas)} filas).")
    else:
        raise RuntimeError("No hay parciales ni resultados disponibles.")

df_consolidado.to_csv(CSV_PATH, index=False)
print(f"Parciales consolidados → {CSV_PATH}")

# — Leer CSV consolidado —
df = pd.read_csv(CSV_PATH)

def calcular_eficiencia(Jc_pos, Jc_neg):
    """
    Calcula la eficiencia del efecto diodo:
     $\eta = |J_{c+} - J_{c-}| / (J_{c+} + J_{c-}) \times 100$ 
    """
    if pd.isna(Jc_pos) or pd.isna(Jc_neg):
        return np.nan
    if (Jc_pos + Jc_neg) == 0:
        return np.nan
    return abs(Jc_pos - Jc_neg) / (Jc_pos + Jc_neg) * 100

# — Calcular eta para cada B en este N —
for B in B_VALORES:
    mask_pos = (df['B_mT'] == B) & (df['polaridad'] == 'positivo')
    mask_neg = (df['B_mT'] == B) & (df['polaridad'] == 'negativo')

    if mask_pos.any() and mask_neg.any():
        Jc1_pos = df.loc[mask_pos, 'Jc1_uA'].values[0]
        Jc1_neg = df.loc[mask_neg, 'Jc1_uA'].values[0]
        Jc2_pos = df.loc[mask_pos, 'Jc2_uA'].values[0]
        Jc2_neg = df.loc[mask_neg, 'Jc2_uA'].values[0]

        eta_c1 = calcular_eficiencia(Jc1_pos, Jc1_neg)
        eta_c2 = calcular_eficiencia(Jc2_pos, Jc2_neg)

        df.loc[mask_pos, 'eta_c1_pct'] = eta_c1
        df.loc[mask_pos, 'eta_c2_pct'] = eta_c2
        df.loc[mask_neg, 'eta_c1_pct'] = eta_c1
        df.loc[mask_neg, 'eta_c2_pct'] = eta_c2
```

```
# — Guardar CSV con eta calculada —
df.to_csv(CSV_PATH, index=False)
print(f"CSV con eficiencias guardado en: {CSV_PATH}")
print(df.to_string(index=False))
```

```
In [ ]: # =====
# CELDA 9 – RESUMEN FINAL CON TIEMPOS
# =====

df = pd.read_csv(CSV_PATH)

exitosos = df[df['estado'] == 'OK']
fallidos = df[df['estado'] != 'OK']

if len(exitosos) > 0:
    idx_rapido = exitosos['tiempo_s'].idxmin()
    idx_lento = exitosos['tiempo_s'].idxmax()
    rapido = exitosos.loc[idx_rapido]
    lento = exitosos.loc[idx_lento]

horas = int(t_N_total // 3600)
minutos = int((t_N_total % 3600) // 60)
segundos = int(t_N_total % 60)

print()
print("=====")
print(f"          RESUMEN – N_PEINE = {N_PEINE}")
print("=====")
print(f" Fin:                {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
print(f" Tiempo N={N_PEINE}:    {t_N_total:.1f}s → {horas}h {minutos}m {segundos}s")
print(f"")
print(f" Trabajos totales:  {len(TRABAJOS)}")
print(f" Exitosos:          {len(exitosos)}/{len(TRABAJOS)}")
print(f" Fallidos:          {len(fallidos)}/{len(TRABAJOS)}")
print(f"")
if len(exitosos) > 0:
    print(f" Tiempo promedio por trabajo: {exitosos['tiempo_s'].mean():.1f}s")
    print(f" Trabajo más rápido: B={rapido['B_mT']}, {rapido['polaridad']} → {rapido['tiempo_s']}s")
    print(f" Trabajo más lento: B={lento['B_mT']}, {lento['polaridad']} → {lento['tiempo_s']}s")
print(f"")
print(f" Resultados en: {CSV_PATH}")
print(f" Parámetros en: {TXT_PATH}")
```

```

print("=====")

if len(fallidos) > 0:
    print("\n TRABAJOS FALLIDOS:")
    for _, row in fallidos.iterrows():
        print(f" B={row['B_mT']}, {row['polaridad']}: {row['estado']}")

# — Añadir tiempos y resultado al TXT de parámetros —
with open(TXT_PATH, "a", encoding="utf-8") as f:
    f.write("\n--- RESULTADO N={} ---\n".format(N_PEINE))
    f.write(f"Fin: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n")
    f.write(f"Tiempo N={N_PEINE}: {t_N_total:.1f}s ({horas}h {minutos}m {segundos}s)\n")
    f.write(f"Exitosos: {len(exitosos)}/{len(TRABAJOS)}\n")
    f.write(f"Fallidos: {len(fallidos)}/{len(TRABAJOS)}\n")

```

```

In [ ]: # =====
# CELDA 10 – GRAFICAR CURVAS V-I DESDE CSV
# Genera (o regenera) las gráficas curva_VI.png
# leyendo los VI_data.csv ya guardados.
# Útil para rehacer plots sin volver a simular.
# =====

# Leer resultados para obtener Jc1/Jc2 si ya están calculados
df_res = pd.read_csv(CSV_PATH) if os.path.exists(CSV_PATH) else pd.DataFrame()

graficas_generadas = 0
graficas_faltantes = []

for pol in POLARIDADES:
    for B in B_VALORES:
        carpeta_B = os.path.join(N_DIR, pol, f"B-{B:.1f}")
        csv_vi_path = os.path.join(carpeta_B, "VI_data.csv")
        png_path = os.path.join(carpeta_B, "curva_VI.png")

        if not os.path.exists(csv_vi_path):
            graficas_faltantes.append(f"{pol}/B-{B:.1f}")
            continue

        df_vi = pd.read_csv(csv_vi_path)
        corrientes = df_vi['current_uA'].values
        voltajes = df_vi['voltaje_V'].values

```



```
# Recuperar Jc1/Jc2 del CSV de resultados si existe
Jc1, Jc2 = None, None
if not df_res.empty:
    mask = (df_res['B_mT'] == B) & (df_res['polaridad'] == pol)
    if mask.any():
        row = df_res.loc[mask].iloc[0]
        Jc1 = row['Jc1_uA'] if not pd.isna(row['Jc1_uA']) else None
        Jc2 = row['Jc2_uA'] if not pd.isna(row['Jc2_uA']) else None

# Generar gráfica
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(corrientes, voltajes, color='blue', linewidth=1)
if Jc1 is not None:
    ax.axvline(Jc1, color='red', linestyle='--', label=f'Jc1 = {Jc1:.2f}  $\mu$ A')
if Jc2 is not None:
    ax.axvline(Jc2, color='orange', linestyle='--', label=f'Jc2 = {Jc2:.2f}  $\mu$ A')
ax.set_xlabel('Corriente ( $\mu$ A)')
ax.set_ylabel('Voltaje promedio (V)')
ax.set_title(f'Curva V-I - N={N_PEINE}, B={B:.1f} mT, {pol}')
ax.grid(True)
ax.legend()
plt.tight_layout()
fig.savefig(png_path, dpi=150)
plt.close(fig)
graficas_generadas += 1

print(f"Gráficas generadas/actualizadas: {graficas_generadas}")
if graficas_faltantes:
    print(f"CSV faltantes (sin graficar): {len(graficas_faltantes)}")
    for f_ in graficas_faltantes:
        print(f" - {f_}")
```

In []: